

SW Maintenance & Evolution Phase 1

Team Members

Andrew Nabil Ragheb	20236018
Youssof Hamed Mohammed	20236058
Bassel Abdou Abdelghafar	20236022
Malek Taha Esmail	20236080

1-GitHub

<https://github.com/malek9y/maintenance.git>

- Created a public GitHub repository named “maintenance” at github.com/malek9y/maintenance initialized with a README, Java .gitignore, and GPL-2.0 license
 - Obtained UniTime source code from the official mirror at github.com/UniTime/unitime and pushed it to the main branch using GitHub Codespaces
 - Invited all team members as collaborators with Write access and each member accepted their invitation via email
 - Created a dev branch as the integration branch where all feature branches get merged before going to main
 - Each team member created their own branch for their assigned feature or bug fix
 - Applied branch protection rules on main requiring at least 1 Pull Request approval before merging with no bypass allowed
 - All changes were submitted through Pull Requests and reviewed and approved by a teammate before merging
 - Updated the README file with team member names, project description, technology stack, change requests table, and Jira board link
 - Integrated SonarCloud for static code analysis by connecting the repository, adding a GitHub Actions workflow file, and adding a sonar-project.properties configuration file pointing to the JavaSource directory
 - Registered all change requests on Jira and linked the board to the GitHub repository through the README and issues were added in github and assigned to team members

2-Description of the Project (Requirements)

2.1 — Overview

UniTime is a comprehensive, open-source educational scheduling system developed collaboratively by universities in North America and Europe. The system is built using Java (Spring Framework, Hibernate ORM), deployed on Apache Tomcat or WildFly application servers, and uses a relational database (MySQL or Oracle) for persistence. The frontend is built using GWT (Google Web Toolkit) and JSP pages.

The primary purpose of UniTime is to help universities coordinate the complex problem of scheduling: assigning courses, instructors, students, rooms, and time slots in a way that minimizes conflicts and satisfies institutional constraints.

2.2 — Functional Requirements

FR-1: Course Timetabling

- The system shall allow departmental scheduling managers to define courses, their sections (called "classes"), and their scheduling constraints.
- The system shall allow scheduling managers to specify instructor preferences, room preferences, time preferences, and distribution preferences (e.g., "Class A must not overlap with Class B").
- The system shall run an automated constraint-satisfaction solver to generate conflict-free timetable solutions.
- The system shall display multiple timetable solutions and allow managers to compare and select the best one.
- The system shall allow manual overrides and interactive editing of the generated timetable.
- The system shall detect and report scheduling conflicts when manual changes are made.

FR-2: Examination Timetabling

- The system shall support scheduling of mid-term and final examinations separately from regular classes.
- The system shall allow definition of exam periods, exam rooms, and seating arrangements.
- The system shall enforce that students who are enrolled in multiple courses do not have overlapping exams.
- The system shall allow instructors and departments to specify exam time preferences.
- The system shall generate examination timetable solutions using a dedicated solver.

FR-3: Student Scheduling

- The system shall allow students to submit course requests through a Scheduling Assistant interface.
- The system shall attempt to enroll students into their requested courses while avoiding conflicts in their individual schedule.
- The system shall support wait-listing: if a class is full, a student can be placed on a wait-list and enrolled if a seat becomes available.
- The system shall support override requests: students may request enrollment beyond the class limit.
- The system shall display a student's personal timetable showing all enrolled classes and exams.

FR-4: Room Management

- The system shall maintain a database of rooms including their capacity, features (e.g., projector, computer lab), and availability.
- The system shall allow rooms to be shared between academic events (classes, exams) and non-academic events (meetings, external bookings).
- The system shall allow administrators to define room availability patterns per academic term.
- The system shall track room usage and flag conflicts.

FR-5: Event Management

- The system shall allow authorized users to book rooms for non-class events (seminars, meetings, etc.).
- The system shall display a unified calendar view of room occupancy.
- The system shall send notifications or approval requests for event bookings.

FR-6: Instructor Management

- The system shall maintain instructor profiles including their associated departments, courses, and preferences.
- The system shall allow instructors to define their own availability and time preferences.
- The system shall support instructor scheduling, assigning instructors to classes based on their qualifications and availability.
- The system shall allow an instructor to have an additional role (e.g., event manager).

FR-7: Data Exchange & Integration

- The system shall support importing and exporting data in XML format (courses, enrollments, rooms, timetables).
- The system shall be integrable with external Student Information Systems (SIS) via configurable data connectors.
- The system shall support rolling timetable data forward from one academic session to the next.

FR-8: User Management & Security

- The system shall support role-based access control with roles including: Administrator, Departmental Manager, Schedule Deputy, Instructor, Student, Event Manager, and Curriculum Manager.
- The system shall authenticate users via UniTime's internal user database or via external authentication (LDAP, CAS, Shibboleth).
- The system shall allow authenticated users to change their password.
- The system shall enforce that users can only access and modify data relevant to their role and department.

FR-9: Reporting & Dashboards

- The system shall provide dashboard views showing scheduling statistics, unassigned classes, conflicts, and solver progress.
- The system shall allow export of reports in PDF or spreadsheet format.
- The system shall display a timetable grid (weekly view) for any room, instructor, or department.

FR-10: Multi-Session / Multi-University Support

- The system shall support multiple academic sessions (semesters/terms) within a single installation.
- The system shall allow multiple departments and universities to use the system concurrently in a distributed manner.

2.3 — Non-Functional Requirements

| NFR-1 | Performance | The solver must produce a feasible timetable for a medium-sized university (500 classes, 200 rooms) within a reasonable time (e.g., under 30 minutes). |

| NFR-2 | Scalability | The system must support up to 25,000 monthly active users concurrently. |

| NFR-3 | Availability | The system must be available during peak scheduling periods (start of semester) with minimal downtime. |

| NFR-4 | Security | All communication must be over HTTPS. Passwords must be stored hashed. Role-based access must be enforced on all pages. |

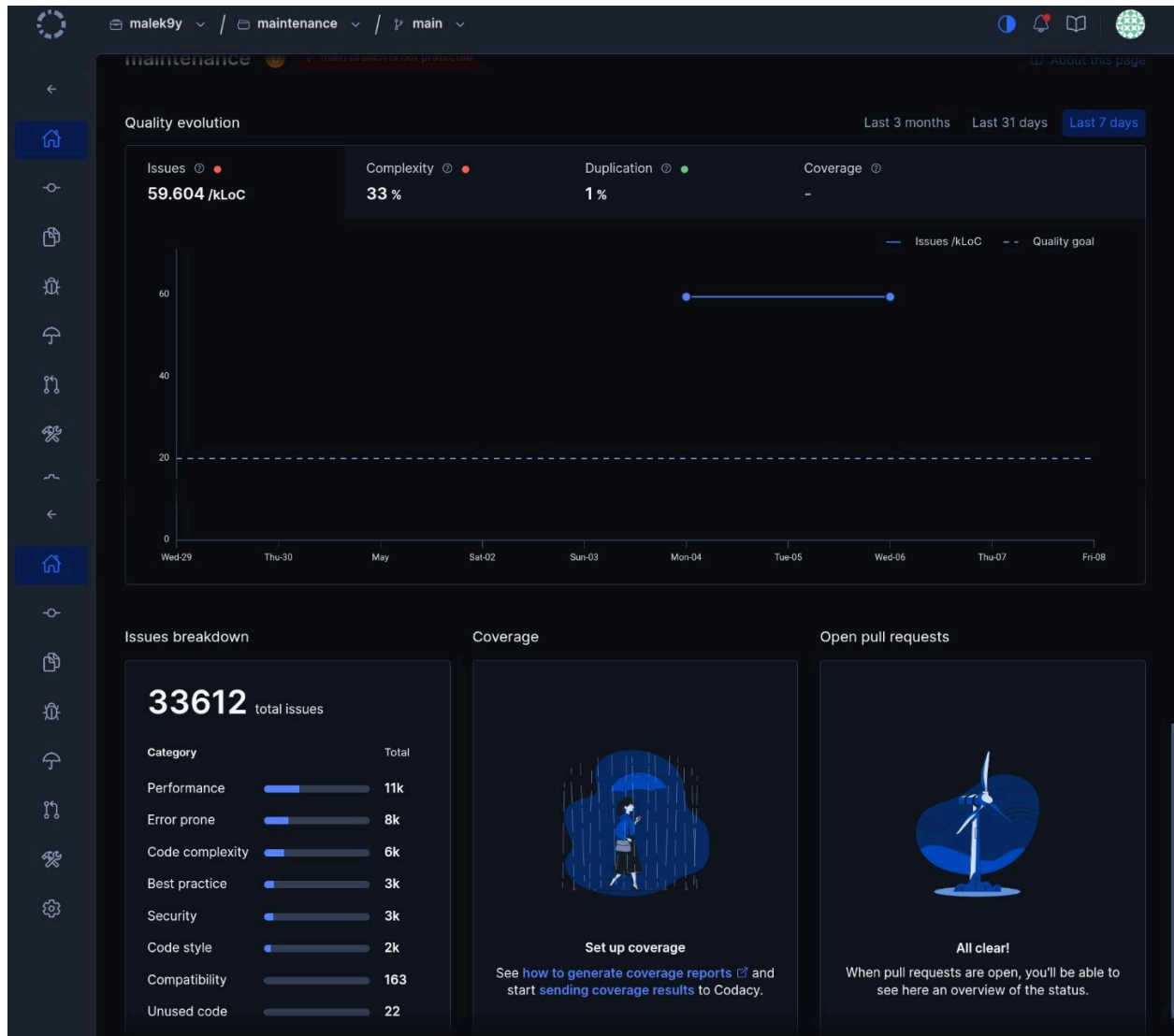
| NFR-5 | Maintainability | The codebase must follow Java coding standards. SonarQube code quality gates must pass. |

| NFR-6 | Portability | The system must run on Java 17+ and support both MySQL and Oracle as database backends. |

| NFR-7 | Usability | All event management pages must be accessible (WAI-ARIA compliant). |

| NFR-8 | Internationalization | The system must support localization. English and Czech translations are provided out of the box. |

3- Code Analysis via Codacy



4-Description for features/bugs

CR-001 — New Feature: Password Strength Validator

Type: New Feature Priority: Medium Assigned to: Youssef

Motivation

The current UniTime password change feature (accessible via Menu → Preferences → Change Password) accepts any password regardless of length or complexity. This poses a significant security risk, as users may set trivially weak passwords. Modern security standards (NIST SP 800-63B) require passwords to meet minimum strength criteria.

Description

Add a server-side password strength validator that enforces the following rules when a user changes their password:

- Minimum 8 characters
- At least 1 uppercase letter
- At least 1 lowercase letter
- At least 1 digit
- At least 1 special character (e.g., !@#\$%^&*)
- Password must not match the username
- New password must differ from the old password

If the password fails validation, return a clear, descriptive error message to the user.

Files to Modify

The password change logic is in the authentication/user management layer:

- `org.unitime.timetable.action.ChangePasswordAction` (or equivalent)
- The corresponding JSP view

CR-002 — Bug Fix: Room Capacity vs. Class Enrollment Validation

Type: Bug Fix Priority: High Assigned to: Malek

Motivation

In the current UniTime system, it is possible for a scheduling manager to manually assign a class to a room whose capacity is smaller than the class's enrollment limit. The solver itself avoids this, but when a manager makes a manual override, there is no validation check. This leads to situations where classes are assigned to rooms that cannot physically accommodate all enrolled students, which is a data integrity and operational safety issue.

Description

Add server-side validation to the class assignment action that:

1. Checks the room capacity of any manually assigned room.
2. Compares it against the class enrollment limit.
3. If the room capacity is less than the class limit, display a warning (or block the assignment, based on configuration).

Root Cause

The validation is missing in the action/service layer that handles manual class-to-room assignment. The solver has this check internally, but the REST/Struts action layer bypasses it for manual edits.

CR-003 — New Feature: Email Notification on Timetable Conflict

Type: New Feature Priority: Medium Assigned to: Andrew

Motivation

When a scheduling conflict is detected (e.g., a room is double-booked, or an instructor is assigned to two overlapping classes), the system currently only highlights the conflict in the UI. However, departmental managers who are not actively using the system at that moment are not notified. An automated email notification would ensure that responsible parties are promptly informed of conflicts that require resolution.

Description

Implement an email notification service that sends an alert email to the relevant departmental scheduling manager whenever a hard scheduling conflict is detected. The email should include:

- Description of the conflict type
- Affected class(es)
- Affected room or instructor
- Time slot
- A direct link to the affected class in UniTime

CR-004 — New Feature: Audit Logging for Timetable Changes

Type: New Feature Priority: High Assigned to: Bassel

Motivation

Currently, UniTime does not maintain a detailed audit trail of who made what changes to the timetable and when. When scheduling disputes arise, there is no reliable way to trace back the history of changes. An audit log is critical for accountability, debugging, and compliance with institutional policies.

Description

Implement an audit logging mechanism that records every significant timetable change event to a database table. Each log entry should capture:

- Timestamp of the change
- Username of the user who made the change
- Type of change (CREATE, UPDATE, DELETE)
- Entity type (Class, Room Assignment, Time Assignment, etc.)
- Entity ID
- Old value (before change)
- New value (after change)
- IP address of the user

5-JIRA

<https://unitime-maintenance.atlassian.net/jira/software/projects/SCRUM/boards/1?atlOrigin=eyJpIjoiNzk5NmFhYzE1NDM2YTlmOTIyM2JmMzgwOWFiYzliLCJwIjoiaSJ9>

The screenshot displays the Jira interface for the 'UniTime Maintenance' project. The left sidebar shows navigation options like 'For you', 'Recent', 'Starred', 'Apps', 'Plans', 'Spaces', 'Recent', 'UniTime Maintenance', 'More spaces', 'Recommended', 'Collect requests', 'Import work', 'Filters', 'Dashboards', 'Teams', and 'Customize sidebar'. The main area shows the 'Board' view with a search bar, a '+ Create' button, and a 'See plans' link. Below the search bar, there are tabs for 'Summary', 'Backlog', 'Board', 'Code', 'Timeline', 'Docs', and 'Forms'. The board itself is divided into four columns: 'TO DO' (2 items), 'IN PROGRESS' (2 items), 'IN REVIEW' (0 items), and 'DONE' (0 items). Each item in the 'TO DO' and 'IN PROGRESS' columns has a title, a description, a status icon, and a label. The 'TO DO' items are 'Send email notification to department manager when scheduling conflict is detected' (SCRUM-3) and 'Implement audit Logging for all time table change events' (SCRUM-4). The 'IN PROGRESS' items are 'Add password strength Validator for user password changes' (SCRUM-1) and 'No Validation when manually assigning a room smaller than class enrollment limit' (SCRUM-2). The 'IN REVIEW' and 'DONE' columns are currently empty.

Search

+ Create

See plans

UniTime Maintenance

Summary Backlog Board Code Timeline Docs Forms

Search bo... Filter

Complete sprint

Group

TO DO 2

Send email notification to department manager when scheduling conflict is detected

SCRUM-3

Implement audit Logging for all time table change events

loggingnew-featuredata...

SCRUM-4

+ Create

IN PROGRESS 2

Add password strength Validator for user password changes

security-new-feature

SCRUM-1

No Validation when manually assigning a room smaller than class enrollment limit

bugvalidationroom-mana...

SCRUM-2

IN REVIEW

DONE